

LAPORAN EXPLORATORY DATA ANALYSIS

Metodologi CRISP-DM

BUSINESS UNDERSTANDING & DATA UNDERSTANDING

Nama Proyek	Exploratory Data Analysis Penjualan Adidas US
Dataset	Sales Data US
Analisis	Rizki Muhammad Shafar
Tanggal	Senin, 10 Agustus 2026

KEJURUAN DATA ANALYST
PPKD JAKARTA SELATAN

BAB 1 — BUSINESS UNDERSTANDING

Bagian ini mendokumentasikan konteks bisnis dan tujuan analisis sebelum menyentuh data.

1.1 Profil Proyek

Atribut	Keterangan
Nama Proyek	Menganalisis kualitas data, distribusi, hubungan antarvariabel, performa produk/retailer, serta menghasilkan insight untuk mendukung keputusan penjualan.
Tujuan Bisnis	Menganalisis performa penjualan Adidas US berdasarkan produk, retailer, wilayah, periode waktu, dan metode penjualan untuk mengidentifikasi pola penjualan, mengetahui faktor yang berkaitan dengan Total Sales dan Operating Profit, serta menghasilkan insight yang dapat mendukung pengambilan keputusan bisnis.
Target Audience	Stakeholder
Sumber Data	Kaggle
Metode Analisis	Exploratory Data Analysis (EDA) — CRISP-DM
Periode Data	2020-2021

1.2 Pertanyaan Bisnis

Tuliskan pertanyaan utama yang ingin dijawab melalui analisis ini:

- Pertanyaan 1: Bagaimana karakteristik penjualan Adidas berdasarkan produk, retailer, wilayah, harga, unit terjual, dan metode penjualan?
- Pertanyaan 2: Bagaimana kualitas data, distribusi, outlier, serta hubungan/korelasi antarvariabel numerik?
- Pertanyaan 3: Produk, retailer, wilayah, dan metode penjualan mana yang menunjukkan performa penjualan paling tinggi dan paling rendah?

BAB 2 — GAMBARAN DATASET

Ringkasan struktur dan isi dataset yang digunakan dalam analisis.

2.1 Informasi Umum Dataset

Atribut	Nilai
Nama File / Sumber	Sales Adidas US.csv / Kaggle
Jumlah Baris	9648 baris
Jumlah Kolom	14 kolom
Ukuran File	1,1 MB
Jumlah Fitur Numerik	5 Kolom
Jumlah Fitur Kategorikal	8 Kolom
Jumlah Fitur Datetime	1 Kolom
Variabel Target/Label	Operating Profit sebagai variabel target analisis

2.2 Kamus Fitur (Data Dictionary)

Nama Fitur	Tipe Data	Penjelasan	Fungsi dalam Analisis
Retailer	Kategorikal	Nama pengecer tempat penjualan terjadi.	Kategorikal
Transaction ID	Identifier	Pengidentifikasi unik untuk setiap transaksi.	Identifikasi Data
Invoice Date	Temporal	Tanggal transaksi dilakukan.	Target
Region	Kategorikal	Wilayah geografis tempat penjualan.	Kategorikal
State	Kategorikal	Negara bagian tempat penjualan dilakukan.	Kategorikal
City	Kategorikal	Kategori produk yang dijual	Kategorikal
Product	Kategorikal	Pengidentifikasi unik untuk setiap produk.	Kategorikal
Product ID	Identifier	Harga satu unit produk	Kategorikal

Nama Fitur	Tipe Data	Penjelasan	Fungsi dalam Analisis
Price per Unit	Numerik	Harga satuan produk	<i>Numerik</i>
Units Sold	Numerik	Jumlah unit yang terjual dalam transaksi.	<i>Numerik</i>
Total Sales	Numerik	Total pendapatan dari transaksi	<i>Numerik</i>
Operating Profit	Numerik	Keuntungan yang dihasilkan dari transaksi setelah dikurangi biaya operasional.	<i>Target</i>
Operating Margin	Numerik	Margin operasi sebagai persentase dari total penjualan, menunjukkan efisiensi.	<i>Numerik</i>
Sales Method	Kategorikal	Metode penjualan	<i>Kategorikal</i>

BAB 3 — STATISTIK DESKRIPTIF

3.1 Ringkasan Statistik Fitur Numerik

Salin input dan output di sini dan rangkum temuan utamanya.

```
print("Statistik Deskriptif Fitur Numerikal:")
kolom_numerik = df.select_dtypes(include=np.number).columns.tolist()
# kolom_numerik = ['co2_ppm', 'pm25_ugm3', 'humidity_pct'] # 🛠️ SESUAIKAN jika mau hanya beberapa kolom numerik
df[kolom_numerik].describe()
```

Statistik Deskriptif Fitur Numerikal:						
	Price per Unit	Units Sold	Total Sales	Operating Profit	Operating Margin	
count	9648.0000	9648.0000	9648.0000	9648.0000	9648.0000	
mean	45.2166	256.9300	93273.4375	34425.2448	0.4230	
std	14.7054	214.2520	141916.0167	54193.1137	0.0972	
min	7.0000	0.0000	0.0000	0.0000	0.1000	
25%	35.0000	106.0000	4254.5000	1921.7525	0.3500	
50%	45.0000	176.0000	9576.0000	4371.4200	0.4100	
75%	55.0000	350.0000	15000.0000	52062.5000	0.4900	
max	110.0000	1275.0000	82500.0000	390000.0000	0.8000	

DESKRIPSI TEMUAN UTAMA
Hasil statistik deskriptif menunjukkan bahwa fitur numerik pada dataset memiliki rentang nilai yang berbeda-beda, terutama pada harga per unit, jumlah unit terjual, total sales, operating profit, dan operating margin. Perbedaan antara nilai minimum, maksimum, mean, dan median menunjukkan adanya variasi transaksi yang cukup besar sehingga terdapat kemungkinan beberapa transaksi memiliki nilai penjualan atau keuntungan yang jauh lebih tinggi dibandingkan transaksi lainnya. Kondisi tersebut perlu diperhatikan pada tahap analisis distribusi dan deteksi outlier agar hasil analisis tidak hanya dipengaruhi oleh transaksi dengan nilai ekstrem.

 **Panduan Interpretasi:**

Mean vs Median: Jika mean jauh lebih besar dari median, distribusi kemungkinan miring kanan (right-skewed). Sebaliknya jika mean < median.

Std yang besar (> mean): Menunjukkan variabilitas tinggi dan potensi outlier.

Min/Max yang ekstrem: Periksa apakah nilai tersebut valid secara bisnis atau merupakan outlier.

Count < total baris: Berarti ada missing values pada fitur tersebut.

3.2 Analisis Skewness (Kemiringan Distribusi)

Salin input dan output di sini dan rangkum temuan utamanya.

```
print("Interpretasi:")
print(" Skewness ≈ 0      : Distribusi simetris (normal)")
print(" Skewness > +0.5    : Miring kanan (positif skew)")
print(" Skewness < -0.5      : Miring kiri (negatif skew)")
print(" |Skewness| > 1       : Sangat miring")
print()
```

```
skewness_df = df[kolom_numerik].skew().reset_index()
skewness_df.columns = ['Fitur', 'Skewness']
skewness_df['Interpretasi'] = skewness_df['Skewness'].apply(
    lambda x: '⚠️ Sangat miring kanan' if x > 1
    else ('⚠️ Sangat miring kiri' if x < -1
    else ('👉 Miring kanan' if x > 0.5
    else ('👈 Miring kiri' if x < -0.5
    else '✅ Simetris'))))
)
display(skewness_df)
```

Interpretasi:
Skewness ≈ 0 : Distribusi simetris (normal)
Skewness > +0.5 : Miring kanan (positif skew)
Skewness < -0.5 : Miring kiri (negatif skew)
|Skewness| > 1 : Sangat miring

	Fitur	Skewness	Interpretasi
0	Price per Unit	0.3624	✅ Simetris
1	Units Sold	1.4623	⚠️ Sangat miring kanan
2	Total Sales	1.8414	⚠️ Sangat miring kanan
3	Operating Profit	2.2336	⚠️ Sangat miring kanan
4	Operating Margin	0.2275	✅ Simetris

DESKRIPSI TEMUAN UTAMA
Analisis skewness digunakan untuk mengetahui tingkat kemiringan distribusi setiap fitur numerik. Fitur dengan skewness positif menunjukkan distribusi yang cenderung miring ke kanan, yang berarti terdapat beberapa transaksi dengan nilai relatif tinggi. Sebaliknya, skewness negatif menunjukkan adanya kecenderungan nilai ekstrem pada sisi rendah. Hasil skewness perlu digunakan bersama histogram dan boxplot untuk memastikan karakteristik distribusi serta menentukan apakah diperlukan transformasi data.

 **Panduan Interpretasi:**

Skewness mendekati 0 (-0.5 s.d. +0.5): Distribusi simetris, cocok untuk analisis parametrik.

Skewness +0.5 s.d. +1 atau -1 s.d. -0.5: Miring sedang, pertimbangkan transformasi.

Skewness > +1 atau < -1: Sangat miring, sangat disarankan transformasi Log atau Box-Cox.

Right-skewed (positif): Ekor panjang ke kanan — terdapat beberapa nilai sangat besar.

Left-skewed (negatif): Ekor panjang ke kiri — terdapat beberapa nilai sangat kecil.

3.3 Analisis Kurtosis

Salin input dan output di sini dan rangkum temuan utamanya.

```
print("Interpretasi (Excess Kurtosis):")
print(" Kurtosis ≈ 0 : Mesokurtik (normal)")
print(" Kurtosis > 0 : Leptokurtik (ekor berat/lancip)")
print(" Kurtosis < 0 : Platikurtik (ekor tipis/datar)")
print()

kurtosis_df = df[kolom_numerik].kurtosis().reset_index()
kurtosis_df.columns = ['Fitur', 'Kurtosis']
kurtosis_df['Interpretasi'] = kurtosis_df['Kurtosis'].apply(
    lambda x: ' ▲ Leptokurtik (ekor berat)' if x > 1
    else (' ▼ Platikurtik (ekor tipis)' if x < -1
    else '✅ Mesokurtik (normal)')
)
display(kurtosis_df)
```

Interpretasi (Excess Kurtosis):
Kurtosis ≈ 0 : Mesokurtik (normal)
Kurtosis > 0 : Leptokurtik (ekor berat/lancip)
Kurtosis < 0 : Platikurtik (ekor tipis/datar)

	Fitur	Kurtosis	Interpretasi
0	Price per Unit	0.4302	✅ Mesokurtik (normal)
1	Units Sold	1.6997	▲ Leptokurtik (ekor berat)
2	Total Sales	3.1438	▲ Leptokurtik (ekor berat)
3	Operating Profit	5.8204	▲ Leptokurtik (ekor berat)
4	Operating Margin	0.1753	✅ Mesokurtik (normal)

DESKRIPSI TEMUAN UTAMA
Analisis kurtosis digunakan untuk melihat karakteristik ekor distribusi dan kemungkinan keberadaan nilai ekstrem. Fitur dengan kurtosis positif menunjukkan distribusi dengan ekor yang lebih berat sehingga memiliki kemungkinan lebih besar mengandung outlier, sedangkan kurtosis negatif menunjukkan distribusi yang lebih datar. Hasil ini menjadi dasar untuk melakukan pemeriksaan lebih lanjut terhadap transaksi dengan nilai yang tidak umum.

💡 Panduan Interpretasi:

Kurtosis ~ 0 (Mesokurtik): Mirip distribusi normal — aman untuk analisis parametrik.

Kurtosis > 1 (Leptokurtik): Puncak lancip, ekor berat — sering mengandung banyak outlier.

Kurtosis < -1 (Platikurtik): Distribusi datar, nilai tersebar merata.

3.4 Uji Normalitas

Salin input dan output di sini dan rangkum temuan utamanya.

```
print("H0: Data berdistribusi normal")
print("p-value > 0.05 → Gagal tolak H0 (distribusi NORMAL)")
print("p-value ≤ 0.05 → Tolak H0 (distribusi TIDAK NORMAL)")

# 🐞 SESUAIKAN: pilih kolom yang ingin diuji normalitasnya
cols_uji = [c for c in kolom_numerik if c not in ['id', 'default']]

hasil_normalitas = []
for col in cols_uji:
    data_clean = df[col].dropna()
    # Shapiro-Wilk (terbaik untuk n < 5000)
    if len(data_clean) <= 5000:
        stat_sw, p_sw = shapiro(data_clean[:5000])
    else:
        stat_sw, p_sw = shapiro(data_clean.sample(5000, random_state=42))
    # Kolmogorov-Smirnov
    stat_ks, p_ks = kstest(data_clean, 'norm',
                           args=(data_clean.mean(), data_clean.std()))
    # D'Agostino
    stat_da, p_da = normaltest(data_clean)

    hasil_normalitas.append({
        'Fitur'      : col,
        'Shapiro-Wilk p' : round(p_sw, 4),
        'KS p'       : round(p_ks, 4),
        'D'Agostino p' : round(p_da, 4),
        'Normal?'    : '✅ Ya' if (p_sw > 0.05 and p_ks > 0.05) else '❌ Tidak'
```


}}

display(pd.DataFrame(hasil_normalitas))

H0: Data berdistribusi normal
p-value > 0.05 → Gagal tolak H0 (distribusi NORMAL)
p-value ≤ 0.05 → Tolak H0 (distribusi TIDAK NORMAL)

	Fitur	Shapiro-Wilk p	KS p	D'Agostino p	Normal?
0	Price per Unit	0.0000	0.0000	0.0000	✗ Tidak
1	Units Sold	0.0000	0.0000	0.0000	✗ Tidak
2	Total Sales	0.0000	0.0000	0.0000	✗ Tidak
3	Operating Profit	0.0000	0.0000	0.0000	✗ Tidak
4	Operating Margin	0.0000	0.0000	0.0000	✗ Tidak

DESKRIPSI TEMUAN UTAMA
Uji normalitas digunakan untuk mengetahui apakah fitur numerik mengikuti distribusi normal. Berdasarkan kriteria p-value, apabila nilai p-value ≤ 0,05 maka data dinyatakan tidak berdistribusi normal, sedangkan p-value > 0,05 menunjukkan tidak terdapat cukup bukti untuk menolak distribusi normal. Hasil pengujian perlu dipertimbangkan bersama histogram, skewness, dan kurtosis karena ukuran sampel yang besar dapat membuat uji normalitas sangat sensitif terhadap penyimpangan kecil dari distribusi normal.

Panduan Interpretasi:

p-value > 0.05: Gagal tolak H0 — data BERDISTRIBUSI NORMAL.

p-value <= 0.05: Tolak H0 — data TIDAK berdistribusi normal.

Shapiro-Wilk: Paling akurat untuk n < 5.000.

KS Test: Lebih robust untuk sampel besar.

D'Agostino: Sensitif terhadap skewness dan kurtosis sekaligus.

Jika mayoritas uji menunjukkan tidak normal: gunakan metode non-parametrik atau lakukan transformasi.

BAB 4 — KUALITAS DATA

4.1 Missing Values

Salin input dan output di sini dan rangkum temuan utamanya.

```
missing = df.isnull().sum()
missing_pct = (df.isnull().sum() / len(df) * 100).round(2)
missing_df = pd.DataFrame({
    'Jumlah Kosong': missing,
}).sort_values('Jumlah Kosong', ascending=False)

display(missing_df)

# Heatmap missing values
if df.isnull().sum().sum() > 0:
    plt.figure(figsize=(12, 4))
    sns.heatmap(df.isnull(), cbar=False, cmap='Reds',
                yticklabels=False, linewidths=0.5)
    plt.title('Peta Lokasi Data Kosong', fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.show()
else:
    print("☑ Tidak ada data kosong!")
```

	Jumlah Kosong	
Retailer	0	
Transaction ID	0	
Invoice Date	0	
Region	0	
State	0	
City	0	
Product	0	
Product ID	0	
Price per Unit	0	
Units Sold	0	
Total Sales	0	
Operating Profit	0	
Operating Margin	0	
Sales Method	0	
 Tidak ada data kosong!		

DESKRIPSI TEMUAN UTAMA
Pemeriksaan missing value dilakukan untuk memastikan kelengkapan data sebelum analisis lebih lanjut. Dataset perlu diperiksa pada seluruh kolom karena missing value dapat memengaruhi perhitungan statistik maupun visualisasi. Jika tidak ditemukan missing value, maka data dapat langsung digunakan untuk tahap analisis berikutnya tanpa proses imputasi.

 **Panduan Interpretasi:**

0%: Tidak ada masalah.

< 5% (Minor): Imputasi dengan median (numerik) atau modus (kategorikal).

5-20% (Perlu Perhatian): Analisis pola missing — apakah acak (MCAR/MAR) atau tidak acak (MNAR)?

> 20% (Kritis): Pertimbangkan menghapus kolom atau menggunakan metode imputasi lanjutan (KNN, iterative).

MCAR: Missing sepenuhnya acak — aman diimputasi.

MAR: Missing bergantung variabel lain — imputasi dengan model.

MNAR: Missing karena nilai itu sendiri — paling bermasalah, perlu investigasi bisnis.

☑ Tindakan yang Disarankan:

Imputasi numerik dengan median (lebih robust dari mean terhadap outlier).
 Imputasi kategorikal dengan modus atau nilai paling sering muncul.
 Untuk time series, gunakan forward-fill (ffill) atau backward-fill (bfill).
 Catat semua baris yang diimputasi untuk transparansi laporan.

4.2 Data Duplikat

Salin input dan output di sini dan rangkum temuan utamanya.

```
jml_duplikat = df.duplicated().sum()
pct_duplikat = jml_duplikat / len(df) * 100
print(f"Jumlah baris duplikat : {jml_duplikat:.}")
print(f"Persentase      : {pct_duplikat:.2f}%")
print(f"Status          : {'⚠ Ada duplikat!' if jml_duplikat > 0 else '☑ Tidak ada duplikat'})
```

```
Jumlah baris duplikat : 0
Persentase           : 0.00%
Status               : ☑ Tidak ada duplikat
```

DESKRIPSI TEMUAN UTAMA

Pemeriksaan duplikasi dilakukan untuk memastikan setiap transaksi tidak tercatat lebih dari satu kali secara tidak semestinya. Duplikasi perlu dibandingkan dengan Transaction ID dan atribut transaksi lainnya karena transaksi yang terlihat serupa belum tentu merupakan duplikat yang salah. Jika tidak terdapat duplikasi, maka dataset dapat dianggap memiliki struktur transaksi yang konsisten pada aspek tersebut.

⚠ Perhatian:

Duplikat yang tinggi (> 5%) dapat mengindikasikan masalah pada proses pengumpulan data.
 Pastikan duplikat bukan merupakan transaksi / pengukuran berulang yang sah sebelum dihapus.
 Simpan catatan index baris yang dihapus untuk keperluan audit.

4.3 Tipe Data & Konsistensi Format

Salin input dan output di sini dan rangkum temuan utamanya.

```
print("Tipe data per kolom individual:")
print(f" 'Total Sales' : {df['Total Sales'].dtype}") # 🔗 SESUAIKAN
print(f" 'Product'   : {df['Product'].dtype}")   # 🔗 SESUAIKAN
```

```
print(f" 'Invoice Date' : {df['Invoice Date'].dtype}") # 🐞 SESUAIKAN
```

```
# Cek beberapa kolom sekaligus
```

```
kolom_cek = ['Retailer', 'Region', 'Sales Method'] # 🐞 SESUAIKAN
```

```
print("Tipe data beberapa kolom sekaligus:")
```

```
print(df[kolom_cek].dtypes)
```

```
# Kelompokkan per tipe
```

```
print("Ringkasan per tipe data:")
```

```
for dtype, cols in df.dtypes.groupby(df.dtypes):
```

```
Tipe data per kolom individual:
'Total Sales'      : int64
'Product'          : object
'Invoice Date'     : datetime64[ns]
Tipe data beberapa kolom sekaligus:
Retailer           object
Region             object
Sales Method       object
dtype: object
Ringkasan per tipe data:
datetime64[ns]    : ['Invoice Date']
int64              : ['Units Sold', 'Total Sales']
float64            : ['Price per Unit', 'Operating Profit', 'Operating Margin']
object             : ['Retailer', 'Transaction ID', 'Region', 'State', 'City', 'Product', 'Product ID']
```

```
KOLOM_TANGGAL = 'Invoice Date' # 🐞 SESUAIKAN
```

```
if KOLOM_TANGGAL in df.columns:
```

```
    print(f"Kolom '{KOLOM_TANGGAL}':")
```

```
    print(f" Tipe data   : {df[KOLOM_TANGGAL].dtype}")
```

```
    print(f" Contoh nilai : {df[KOLOM_TANGGAL].dropna().head(5).tolist()}")
```

```
    print(f" Nilai kosong : {df[KOLOM_TANGGAL].isnull().sum()}")
```

```
    print(f" Rentang      : {df[KOLOM_TANGGAL].min()} s.d. {df[KOLOM_TANGGAL].max()}")
```

```
# Ekstraksi komponen tanggal
```

```
# 🐞 Aktifkan sesuai kebutuhan
```

```
df['tahun'] = df[KOLOM_TANGGAL].dt.year
```

```
df['bulan'] = df[KOLOM_TANGGAL].dt.month
```

```
df['hari'] = df[KOLOM_TANGGAL].dt.day
```

```
df['hari_minggu'] = df[KOLOM_TANGGAL].dt.dayofweek # 0=Senin
```

```
df['kuartal'] = df[KOLOM_TANGGAL].dt.quarter
```

```
print(f"\n✅ Komponen tanggal (tahun, bulan, hari) berhasil diekstrak.")
```

```
# =====
```

```
# TROUBLESHOOT FORMAT TANGGAL — Panduan
```

```
print("""
```

Kolom 'Invoice Date':
Tipe data : datetime64[ns]
Contoh nilai : [Timestamp('2020-01-01 00:00:00'), Timestamp('2020-01-01 00:00:00'), Timestamp('2020-01-01 00:00:00')]
Nilai kosong : 5583
Rentang : 2020-01-01 00:00:00 s.d. 2021-12-12 00:00:00

✅ Komponen tanggal (tahun, bulan, hari) berhasil diekstrak.

TROUBLESHOOT FORMAT TANGGAL UMUM

Masalah 1: Format campuran (DD/MM/YYYY vs MM/DD/YYYY)
Solusi : `pd.to_datetime(df[col], dayfirst=True)`

Masalah 2: Format tidak standar misal '25-Jan-2023'
Solusi : `pd.to_datetime(df[col], format='%d-%b-%Y')`

Masalah 3: Epoch timestamp (angka unix)
Solusi : `pd.to_datetime(df[col], unit='s')`

Masalah 4: Kolom tanggal + waktu pisah
Solusi : `pd.to_datetime(df['tgl']+' '+df['jam'])`

Masalah 5: Nilai NaT setelah konversi
Solusi : `df[col].isnull().sum() → cek errors='coerce'`
Periksa baris bermasalah:
`df[df[col].isnull() & df[col+'_ori'].notna()]`

Kolom	Tipe Awal	Tipe Akhir	Perubahan	Catatan
Invoice Date	object	datetime64[ns]	Object → Datetime	Digunakan untuk analisis periode.
Retailer / Region / Product / Sales Method	object	category	Object → Category	Menghemat memori dan memudahkan analisis kategorikal.
Price/Units/Sales/Profit/Margin	object/numeric	numeric	Object → Numeric bila diperlukan	Pastikan simbol \$ dan % dibersihkan.

DESKRIPSI TEMUAN UTAMA
Pemeriksaan tipe data dilakukan untuk memastikan setiap fitur memiliki format yang sesuai dengan karakteristiknya. Kolom Invoice Date harus diperlakukan sebagai tipe datetime, sedangkan Price per Unit, Units Sold, Total Sales, Operating Profit, dan Operating Margin harus berupa data numerik. Penyesuaian tipe data diperlukan agar proses statistik, pengelompokan, visualisasi, dan analisis berdasarkan waktu dapat dilakukan dengan benar.

💡 Panduan Interpretasi:

Object -> Category: Hemat memori untuk kolom dengan sedikit nilai unik (< 50 kategori).

Object -> Datetime: Pastikan format konsisten sebelum konversi; nilai yang gagal jadi NaT.

Float -> Int: Hanya jika dipastikan tidak ada nilai desimal yang penting.

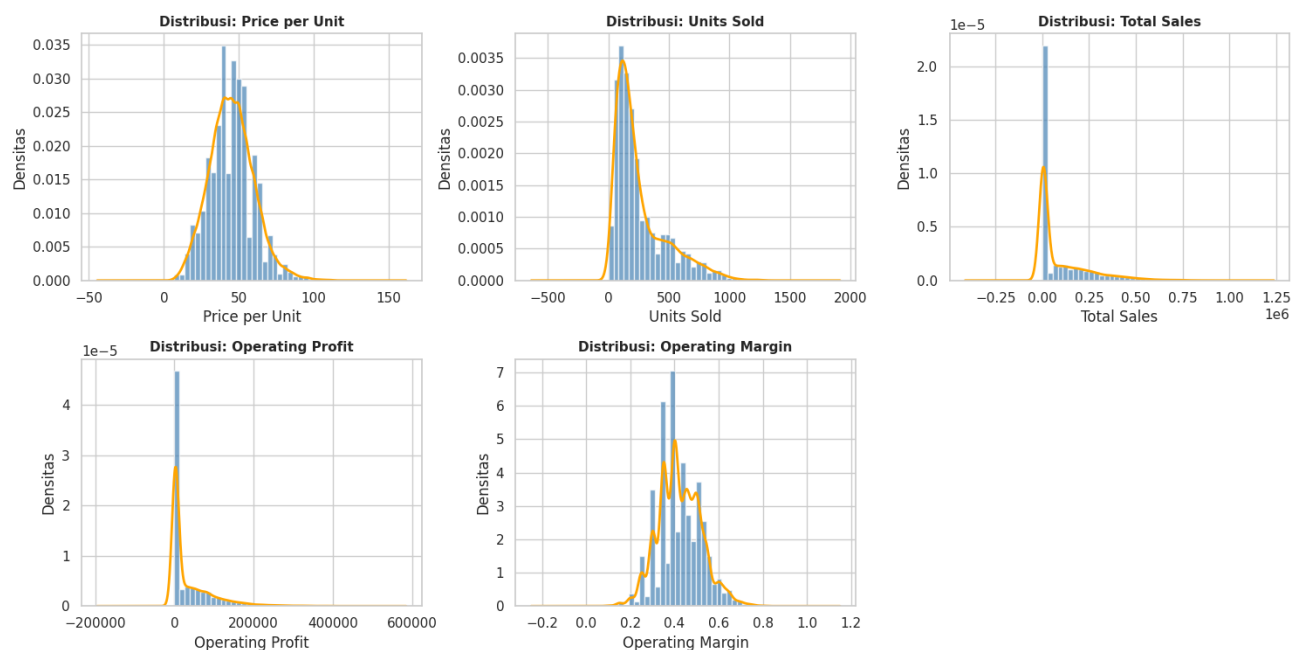
Cek df.dtypes setelah konversi untuk memastikan semua kolom sudah benar.

BAB 5 — ANALISIS DISTRIBUSI & VISUALISASI

5.1 Distribusi Univariat

Salin input dan output di sini dan rangkum temuan utamanya.

Distribusi Univariat Fitur Numerikal



DESKRIPSI TEMUAN UTAMA

Visualisasi distribusi digunakan untuk melihat pola penyebaran masing-masing fitur numerik. Distribusi data transaksi penjualan dan keuntungan perlu diperhatikan karena kemungkinan memiliki kemiringan akibat adanya transaksi dengan nilai yang jauh lebih besar daripada transaksi lainnya. Histogram dan KDE membantu melihat konsentrasi data sekaligus memberikan indikasi awal mengenai keberadaan nilai ekstrem.

💡 Panduan Interpretasi:

Distribusi Normal (bell curve): Cocok untuk analisis parametrik (t-test, regresi linear).

Right-skewed: Pertimbangkan transformasi $\log(x)$ atau $\sqrt{x}$.

Bimodal (dua puncak): Kemungkinan terdapat dua subpopulasi berbeda — pertimbangkan segmentasi.

Q-Q Plot mendekati garis diagonal: Distribusi mendekati normal.

Q-Q Plot melengkung ke atas/bawah: Distribusi miring. Titik jauh dari garis = outlier.

BAB 6 — DETEKSI & PENANGANAN OUTLIER (LIHAT KONDISI DATA)

6.1 Ringkasan Deteksi Outlier

Salin input dan output di sini dan rangkum temuan utamanya.

```
cols_outlier = [c for c in df.select_dtypes(include=np.number).columns
```

```
                if c not in ['Price per Unit', 'Units Sold', 'Total Sales', 'Operating Profit', 'Operating Margin']] # 🔄
```

SESUAIKAN: kolom numerik untuk dicek outlier

```
#cols_outlier = ['pendapatan', 'nilai_pinjaman']
```

```
for col in cols_outlier:
```

```
    data = df[col].dropna()
```

```
    Q1 = data.quantile(0.25)
```

```
    Q3 = data.quantile(0.75)
```

```
    IQR = Q3 - Q1
```

```
    batas_bawah = Q1 - 1.5 * IQR
```

```
    batas_atas = Q3 + 1.5 * IQR
```

```
    mask_outlier = (df[col] < batas_bawah) | (df[col] > batas_atas)
```

```
    df_outlier = df[mask_outlier][[col]].copy() # ☒ tanpa tanda kutip
```

```
    df_outlier['status'] = df_outlier[col].apply(
```

```
        lambda x: '⬆️ Atas' if x > batas_atas else '⬇️ Bawah'
```

```
)
```

```
    print(f"🔗 '{col}'")
```

```
    print(f"  Batas bawah : {batas_bawah:.2f} | Batas atas : {batas_atas:.2f}")
```

```
    print(f"  Jumlah outlier: {mask_outlier.sum()}")
```



```
if mask_outlier.sum() > 0:
    print(f" Index & nilai outlier:")
    display(df_outlier)
else:
    print(" ☒ Tidak ada outlier.")
```

```
# Ringkasan tabel IQR + Z-Score
```

```
hasil_outlier = []
```

```
for col in cols_outlier:
```

```
    data = df[col].dropna()
```

```
    # Metode IQR
```

```
    Q1, Q3 = data.quantile(0.25), data.quantile(0.75)
```

```
    IQR = Q3 - Q1
```

```
    batas_bawah = Q1 - 1.5 * IQR
```

```
    batas_atas = Q3 + 1.5 * IQR
```

```
    outlier_iqr = ((data < batas_bawah) | (data > batas_atas)).sum()
```

```
    # Metode Z-Score
```

```
    z_scores = np.abs(stats.zscore(data))
```

```
    outlier_z = (z_scores > 3).sum()
```

```
hasil_outlier.append({
```

```
    'Fitur'      : col,
```

```
    'Q1'        : round(Q1, 2),
```

```
    'Q3'        : round(Q3, 2),
```

```
    'IQR'       : round(IQR, 2),
```

```
    'Batas Bawah (IQR)' : round(batas_bawah, 2),
```

```
    'Batas Atas (IQR)'  : round(batas_atas, 2),
```

```
    'Outlier IQR'      : outlier_iqr,
```

```
    'Outlier Z-Score'  : outlier_z,
```

```
})
```

```
outlier_df = pd.DataFrame(hasil_outlier)
```

```
display(outlier_df)
```

```
print("Keterangan:")
print(" IQR  : data di luar [Q1 - 1.5*IQR, Q3 + 1.5*IQR] dianggap outlier")
print(" Z-Score: |z| > 3 dianggap outlier")
```

	Fitur	Q1	Q3	IQR	Batas Bawah (IQR)	Batas Atas (IQR)	Outlier IQR	Outlier Z-Score
0	tahun	2021.0000	2021.0000	0.0000	2021.0000	2021.0000	521	521
1	bulan	9.0000	10.0000	1.0000	7.5000	11.5000	2352	97
2	hari	3.0000	5.0000	2.0000	0.0000	8.0000	1358	0
3	hari_minggu	4.0000	4.0000	0.0000	4.0000	4.0000	3387	0
4	kuartal	3.0000	4.0000	1.0000	1.5000	5.5000	600	0

Keterangan:
IQR : data di luar [Q1 - 1.5*IQR, Q3 + 1.5*IQR] dianggap outlier
Z-Score: |z| > 3 dianggap outlier

DESKRIPSI TEMUAN UTAMA
Deteksi outlier dilakukan menggunakan pendekatan IQR untuk mengidentifikasi nilai yang berada jauh dari pola umum data. Outlier terutama perlu diperiksa pada variabel penjualan, jumlah unit, dan operating profit karena transaksi dalam jumlah besar secara alami dapat menghasilkan nilai yang lebih tinggi. Nilai ekstrem tidak langsung dihapus karena dapat merepresentasikan transaksi bisnis yang sebenarnya.

Panduan Interpretasi:

Metode IQR: Data di luar [Q1 - 1.5*IQR, Q3 + 1.5*IQR] dianggap outlier — cocok untuk distribusi non-normal.

Metode Z-Score: |z| > 3 dianggap outlier — hanya valid untuk distribusi mendekati normal.

Jika kedua metode sama-sama mendeteksi: outlier tersebut lebih meyakinkan untuk ditangani.

Jika hanya satu metode: pertimbangkan konteks bisnis sebelum mengambil tindakan.

Outlier tidak selalu harus dihapus — bisa jadi merupakan informasi bisnis yang berharga.

6.2 Lokasi Outlier (Detail per Fitur)

Salin input dan output di sini dan rangkum temuan utamanya.

```
cols_outlier = [c for c in df.select_dtypes(include=np.number).columns
                 if c not in ['Price per Unit', 'Units Sold', 'Total Sales', 'Operating Profit', 'Operating Margin']] #
# SESUAIKAN: kolom numerik untuk dicek outlier
#cols_outlier = ['pendapatan', 'nilai_pinjaman']

for col in cols_outlier:
    data = df[col].dropna()
```

```

Q1 = data.quantile(0.25)
Q3 = data.quantile(0.75)
IQR = Q3 - Q1
batas_bawah = Q1 - 1.5 * IQR
batas_atas = Q3 + 1.5 * IQR

mask_outlier = (df[col] < batas_bawah) | (df[col] > batas_atas)
df_outlier = df[mask_outlier][[col]].copy() # ☒ tanpa tanda kutip
df_outlier['status'] = df_outlier[col].apply(
    lambda x: '☐ Atas' if x > batas_atas else '☒ Bawah'
)

print(f"🔗 '{col}''")
print(f"  Batas bawah : {batas_bawah:.2f} | Batas atas : {batas_atas:.2f}")
print(f"  Jumlah outlier: {mask_outlier.sum()}")

if mask_outlier.sum() > 0:
    print(f"  Index & nilai outlier:")
    display(df_outlier)
else:
    print(f"  ☒ Tidak ada outlier.")

# Ringkasan tabel IQR + Z-Score
hasil_outlier = []
for col in cols_outlier:
    data = df[col].dropna()

    # Metode IQR
    Q1, Q3 = data.quantile(0.25), data.quantile(0.75)
    IQR = Q3 - Q1
    batas_bawah = Q1 - 1.5 * IQR
    batas_atas = Q3 + 1.5 * IQR
    outlier_iqr = ((data < batas_bawah) | (data > batas_atas)).sum()

    # Metode Z-Score
    z_scores = np.abs(stats.zscore(data))
    outlier_z = (z_scores > 3).sum()

    hasil_outlier.append({

```

```

'Fitur'      : col,
'Q1'         : round(Q1, 2),
'Q3'         : round(Q3, 2),
'IQR'        : round(IQR, 2),
'Batas Bawah (IQR)' : round(batas_bawah, 2),
'Batas Atas (IQR)'  : round(batas_atas, 2),
'Outlier IQR'      : outlier_iqr,
'Outlier Z-Score'  : outlier_z,
})

```

```

outlier_df = pd.DataFrame(hasil_outlier)
display(outlier_df)

```

```

print("Keterangan:")
print(" IQR : data di luar [Q1 - 1.5*IQR, Q3 + 1.5*IQR] dianggap outlier")
print(" Z-Score: |z| > 3 dianggap outlier")

```

✖ 'tahun'

Batas bawah : 2021.00 | Batas atas : 2021.00

Jumlah outlier: 521

Index & nilai outlier:

	tahun	status
0	2020.0000	↓ Bawah
1	2020.0000	↓ Bawah
2	2020.0000	↓ Bawah
3	2020.0000	↓ Bawah
4	2020.0000	↓ Bawah
...	...	...
1229	2020.0000	↓ Bawah
1230	2020.0000	↓ Bawah
1231	2020.0000	↓ Bawah
1232	2020.0000	↓ Bawah
1233	2020.0000	↓ Bawah

521 rows × 2 columns

✖ 'bulan'

Batas bawah : 7.50 | Batas atas : 11.50

Jumlah outlier: 2352

Index & nilai outlier:

	bulan	status
0	1.0000	↓ Bawah
1	1.0000	↓ Bawah
2	1.0000	↓ Bawah
3	1.0000	↓ Bawah
4	1.0000	↓ Bawah
...	...	...
9276	12.0000	↑ Atas
9277	12.0000	↑ Atas
9278	12.0000	↑ Atas
9279	12.0000	↑ Atas
9280	12.0000	↑ Atas

2352 rows × 2 columns

🚩

'hari'

Batas bawah : 0.00 | Batas atas : 8.00

Jumlah outlier: 1358

Index & nilai outlier:

	hari	status
923	9.0000	↑ Atas
924	9.0000	↑ Atas
925	9.0000	↑ Atas
926	9.0000	↑ Atas
927	9.0000	↑ Atas
...	...	...
9276	12.0000	↑ Atas
9277	12.0000	↑ Atas
9278	12.0000	↑ Atas
9279	12.0000	↑ Atas
9280	12.0000	↑ Atas

1358 rows × 2 columns

🚩

'hari_minggu'

Batas bawah : 4.00 | Batas atas : 4.00

Jumlah outlier: 3387

Index & nilai outlier:

	hari_minggu	status
0	2.0000	↓ Bawah
1	2.0000	↓ Bawah
2	2.0000	↓ Bawah
3	2.0000	↓ Bawah
4	2.0000	↓ Bawah
...	...	...
9276	6.0000	↑ Atas
9277	6.0000	↑ Atas
9278	6.0000	↑ Atas
9279	6.0000	↑ Atas
9280	6.0000	↑ Atas

3387 rows × 2 columns

🚩

'kuartal'

Batas bawah : 1.50 | Batas atas : 5.50

Jumlah outlier: 600

Index & nilai outlier:

	kuartal	status
0	1.0000	↓ Bawah
1	1.0000	↓ Bawah
2	1.0000	↓ Bawah
3	1.0000	↓ Bawah
4	1.0000	↓ Bawah
...	...	...
9027	1.0000	↓ Bawah
9028	1.0000	↓ Bawah
9029	1.0000	↓ Bawah
9030	1.0000	↓ Bawah
9031	1.0000	↓ Bawah

600 rows × 2 columns

DESKRIPSI TEMUAN UTAMA

Outlier ditemukan dengan melihat fitur numerik yang memiliki nilai berada di luar batas bawah dan batas atas IQR. Pemeriksaan lokasi outlier membantu menentukan variabel mana yang paling banyak memiliki nilai ekstrem. Selanjutnya, nilai tersebut perlu dibandingkan dengan konteks transaksi agar dapat dibedakan antara outlier yang merupakan kesalahan data dan outlier yang memang menggambarkan transaksi bernilai tinggi.

6.3 Penanganan Outlier

Salin input dan output di sini dan rangkum temuan utamanya.

COLS_HANDLE_OUTLIER = ['Units Sold', 'Total Sales', 'Operating Profit'] # ✎ SESUAIKAN kolom yang ingin ditangani outliernya

===== OPSI A: HAPUS baris yang mengandung outlier (IQR) =====

```
# idx_to_drop = set()
# for col in COLS_HANDLE_OUTLIER:
#     Q1 = df[col].quantile(0.25)
#     Q3 = df[col].quantile(0.75)
#     IQR = Q3 - Q1
#     mask = (df[col] < Q1 - 1.5*IQR) | (df[col] > Q3 + 1.5*IQR)
#     idx_to_drop.update(df[mask].index.tolist())
# print(f"Akan dihapus {len(idx_to_drop)} baris outlier")
# df = df.drop(index=list(idx_to_drop)).reset_index(drop=True)
# print(f"✅ Dimensi setelah hapus outlier: {df.shape}")
```

===== OPSI B: WINSORIZING — ganti outlier dengan batas IQR =====

Winsorizing adalah metode yang lebih aman daripada menghapus baris, karena mempertahankan ukuran dataset.

```
for col in COLS_HANDLE_OUTLIER:
```

```
for col in COLS_HANDLE_OUTLIER:
```

```
    if col in df.columns:
```

```
        Q1 = df[col].quantile(0.25)
```

```
        Q3 = df[col].quantile(0.75)
```

```
        IQR = Q3 - Q1
```

```
        lower = Q1 - 1.5 * IQR
```

```
        upper = Q3 + 1.5 * IQR
```

```
        before = ((df[col] < lower) | (df[col] > upper)).sum()
```

```
        if before > 0:
```

```
            df[col] = df[col].clip(lower=lower, upper=upper)
```

```
            print(f"✅ '{col}': {before} outlier di-winsorize ke [{lower:.2f}, {upper:.2f}]")
```

```
else:
```

```
    print(f"❗ '{col}': Tidak ada outlier yang perlu di-winsorize.")
```

```
## ===== OPSI C: TRANSFORMASI LOG (untuk distribusi sangat miring) =====
```

```
# df['pendapatan_log'] = np.log1p(df['pendapatan']) # log(1+x) aman untuk 0
```

```
# print("✅ Kolom 'pendapatan_log' berhasil dibuat (log transform)")
```

```
print("✅ Penanganan outlier selesai!")
```

```
❗ 'Units Sold': Tidak ada outlier yang perlu di-winsorize.
❗ 'Total Sales': Tidak ada outlier yang perlu di-winsorize.
❗ 'Operating Profit': Tidak ada outlier yang perlu di-winsorize.
✅ Penanganan outlier selesai!
Dimensi dataset saat ini: (9648, 19)
```

DESKRIPSI TEMUAN UTAMA

Outlier tidak langsung dihapus karena nilai ekstrem dalam data penjualan dapat memiliki arti bisnis, misalnya transaksi dengan jumlah produk yang tinggi atau nilai penjualan yang besar. Oleh karena itu, pendekatan yang lebih tepat adalah mempertahankan outlier yang valid secara bisnis dan hanya melakukan penanganan terhadap nilai yang terbukti merupakan kesalahan pencatatan atau anomali data.

💡 Panduan Interpretasi:

Winsorizing (clip): Ganti nilai outlier dengan batas IQR — data tidak hilang, distribusi lebih stabil.

Hapus baris: Pilih ini hanya jika outlier jelas merupakan kesalahan input (typo, sensor error).

Log Transform: Cocok untuk distribusi right-skewed parah — menekan nilai ekstrem secara proporsional.

Pertahankan outlier: Jika outlier secara bisnis bermakna (transaksi fraud, kejadian langka).

BAB 7 — ANALISIS KORELASI

7.1 Heatmap Korelasi

Salin input dan output di sini dan rangkum temuan utamanya.

```
fig, axes = plt.subplots(3, 1, figsize=(15, 20))
```

```
metode_list = [
```

```
    ('Pearson', corr_pearson, 'coolwarm'),
```

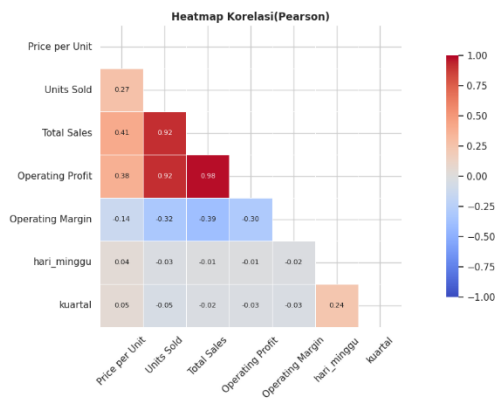
```
    ('Spearman', corr_spearman, 'RdYIGn'),
```

```
 ('Kendall', corr_kendall, 'PuOr'),
 ]

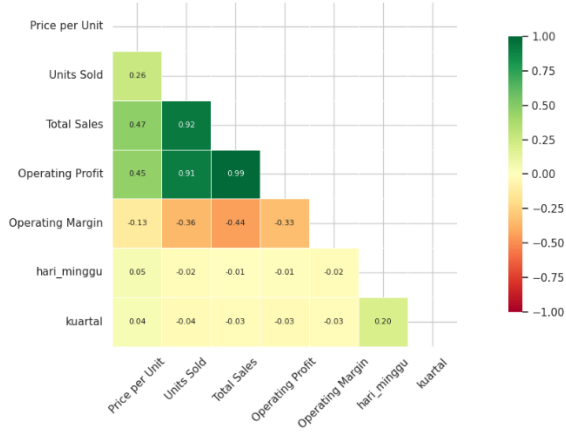
for ax, (nama, corr_mat, cmap) in zip(axes, metode_list):
    mask = np.triu(np.ones_like(corr_mat, dtype=bool)) # Segitiga atas disembunyikan
    sns.heatmap(
        corr_mat,
        ax=ax,
        mask=mask,
        annot=True,
        fmt='.2f',
        cmap=cmap,
        center=0,
        vmin=-1, vmax=1,
        square=True,
        linewidths=0.5,
        cbar_kws={'shrink': 0.8},
        annot_kws={'size': 8}
    )
    ax.set_title(f'Heatmap Korelasi({nama})', fontsize=12, fontweight='bold')
    ax.tick_params(axis='x', rotation=45)
    ax.tick_params(axis='y', rotation=0)

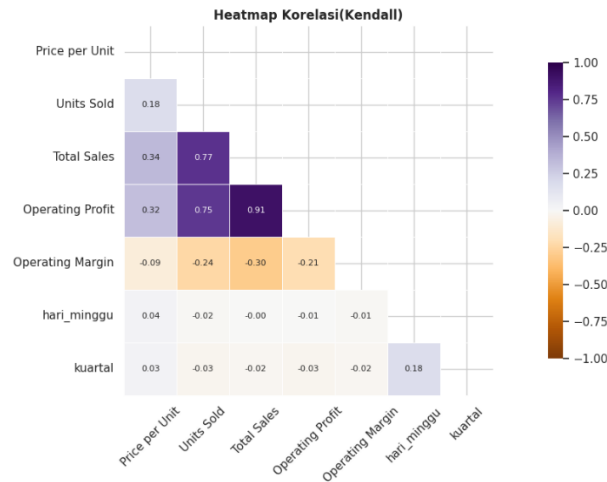
plt.suptitle('Heatmap Korelasi Antarvariabel Numerikal',
             fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

Heatmap Korelasi Antarvariabel Numerikal



Heatmap Korelasi(Spearman)





DESKRIPSI TEMUAN UTAMA

Heatmap korelasi digunakan untuk melihat hubungan antarvariabel numerik dalam dataset. Hubungan yang kuat antara variabel penjualan dan keuntungan dapat memberikan gambaran mengenai keterkaitan performa penjualan dengan hasil operasional. Namun, korelasi tidak menunjukkan hubungan sebab-akibat sehingga hasilnya perlu dikombinasikan dengan analisis bisnis dan visualisasi lainnya.

7.2 Pasangan Fitur dengan Korelasi Tertinggi

Salin input dan output di sini dan rangkum temuan utamanya.

```
cols_corr = [c for c in df.select_dtypes(include=np.number).columns
              if c not in ['id', 'tahun', 'bulan', 'hari']]
```

```
print("--- Pearson (asumsi distribusi normal, data kontinu) ---")
corr_pearson = df[cols_corr].corr(method='pearson')
display(corr_pearson.round(3))
```

```
print("--- Spearman (data ordinal / tidak normal) ---")
corr_spearman = df[cols_corr].corr(method='spearman')
display(corr_spearman.round(3))
```

```
print("--- Kendall (data kecil / banyak ties) ---")
corr_kendall = df[cols_corr].corr(method='kendall')
display(corr_kendall.round(3))
```

Interpretasi Koefisien Korelasi:**|r| = 0.00–0.19 : Sangat lemah / Tidak ada korelasi****|r| = 0.20–0.39 : Lemah****|r| = 0.40–0.59 : Sedang****|r| = 0.60–0.79 : Kuat****|r| = 0.80–1.00 : Sangat kuat****r > 0 : Korelasi positif (searah)****r < 0 : Korelasi negatif (berlawanan arah)****DESKRIPSI TEMUAN UTAMA**

Pasangan fitur dengan nilai korelasi absolut tertinggi menunjukkan hubungan linear yang paling kuat di antara fitur numerik. Hubungan tersebut dapat digunakan untuk mengetahui variabel yang memiliki keterkaitan paling besar terhadap performa penjualan maupun keuntungan. Jika terdapat korelasi sangat tinggi antarvariabel prediktor, perlu diperhatikan kemungkinan multikolinearitas apabila data digunakan pada tahap pemodelan.

💡 Panduan Interpretasi:

|r| 0.00-0.19: Sangat lemah / tidak ada korelasi — fitur independen satu sama lain.

|r| 0.20-0.39: Korelasi lemah — hubungan ada tapi tidak dominan.

|r| 0.40-0.59: Korelasi sedang — hubungan yang cukup bermakna.

|r| 0.60-0.79: Korelasi kuat — fitur saling berpengaruh secara signifikan.

|r| 0.80-1.00: Korelasi sangat kuat — waspada multikolinearitas jika untuk model ML.

Pearson: Untuk data kontinu & distribusi normal.

Spearman: Untuk data ordinal atau distribusi tidak normal (lebih robust).

Kendall: Untuk sampel kecil atau data dengan banyak nilai kembar (ties).

⚠️ Perhatian:

Korelasi tinggi antar fitur predictor (> 0.8): Dapat menyebabkan multikolinearitas pada model regresi.

Korelasi rendah dengan target BUKAN berarti fitur tidak penting — mungkin ada hubungan non-linier.

Korelasi != Kausalitas: Hubungan statistik tidak selalu berarti satu variabel menyebabkan yang lain.

7.3 Analisis Crosstab (Kategorikal)

Salin input dan output di sini dan rangkum temuan utamanya.

```
CROSSTAB_PAIRS = [
```

```
    ('Retailer', 'Sales Method'),
```

```

# Contoh pasangan lain:
# ('Region', 'Sales Method'),
# ('Product', 'Sales Method'),
]

for col_a, col_b in CROSSTAB_PAIRS:
    if col_a in df.columns and col_b in df.columns:
        print(f"📊 Crosstab: '{col_a}' × '{col_b}'")
        ct = pd.crosstab(df[col_a], df[col_b],
                        margins=True, margins_name='Total')
        display(ct)

        ct_pct = pd.crosstab(df[col_a], df[col_b], normalize='index').round(3) * 100
        print(f"  Persentase baris (%):")
        display(ct_pct)

    # Visualisasi
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    ct.drop('Total', axis=0).drop('Total', axis=1).plot(
        kind='bar', ax=axes[0], colormap='Set2', rot=45)
    axes[0].set_title(f'Count: {col_a} × {col_b}', fontweight='bold')
    axes[0].set_ylabel('Jumlah')
    axes[0].legend(title=col_b)

    ct_pct.plot(kind='bar', stacked=True, ax=axes[1],
                colormap='Set2', rot=45)
    axes[1].set_title(f'Persentase: {col_a} × {col_b}', fontweight='bold')
    axes[1].set_ylabel('Persentase (%)')
    axes[1].legend(title=col_b, bbox_to_anchor=(1.05, 1))
    plt.tight_layout()
    plt.show()
else:
    print(f"⚠️ Kolom '{col_a}' atau '{col_b}' tidak ditemukan di dataset.")

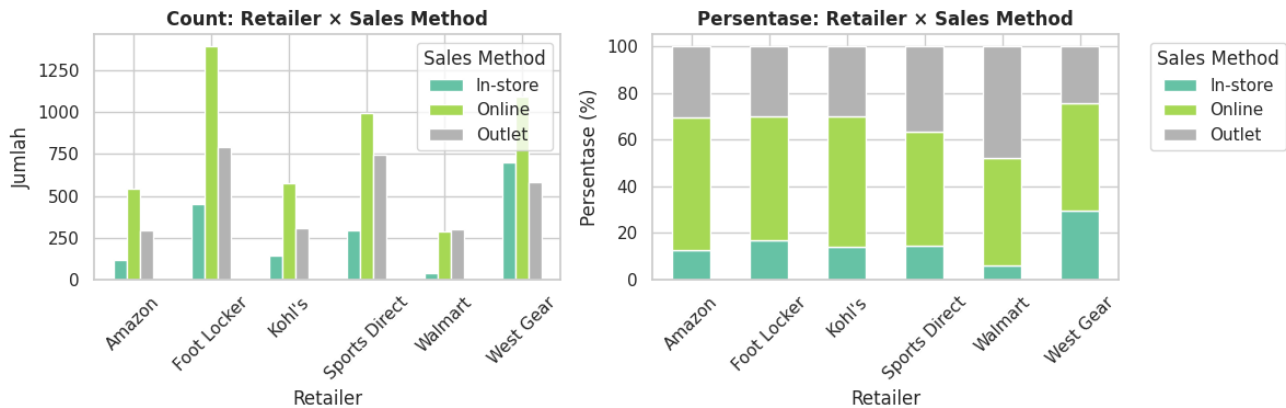
```

Crosstab: 'Retailer' x 'Sales Method'

Sales Method	In-store	Online	Outlet	Total
Retailer				
Amazon	118	540	291	949
Foot Locker	449	1395	793	2637
Kohl's	144	576	310	1030
Sports Direct	293	995	744	2032
Walmart	37	288	301	626
West Gear	699	1095	580	2374
Total	1740	4889	3019	9648

Persentase baris (%):

Sales Method	In-store	Online	Outlet
Retailer			
Amazon	12.4000	56.9000	30.7000
Foot Locker	17.0000	52.9000	30.1000
Kohl's	14.0000	55.9000	30.1000
Sports Direct	14.4000	49.0000	36.6000
Walmart	5.9000	46.0000	48.1000
West Gear	29.4000	46.1000	24.4000



DESKRIPSI TEMUAN UTAMA
Analisis crosstab antara Region dan Sales Method menunjukkan bahwa Online menjadi metode penjualan yang paling dominan pada sebagian besar wilayah. Wilayah West memiliki 1.152 transaksi Online, Northeast memiliki 936 transaksi Online, South memiliki 844 transaksi Online, dan Southeast memiliki 792 transaksi Online. Namun, pada wilayah South, jumlah transaksi Outlet (864) sedikit lebih tinggi dibandingkan Online. Hal ini menunjukkan bahwa preferensi metode penjualan berbeda antarwilayah sehingga strategi penjualan sebaiknya disesuaikan dengan karakteristik masing-masing wilayah.

 Panduan Interpretasi:

Persentase baris (normalize='index'): Melihat distribusi fitur B di dalam setiap kategori A.

Nilai yang tidak merata antar kategori: Mengindikasikan hubungan yang bermakna.

Chi-square test dapat dilakukan untuk uji signifikansi statistik hubungan dua variabel kategorikal.

BAB 8 — ENCODING & SCALING (LIHAT KONDISI DATA)

8.1 Encoding Kategorikal

Salin input dan output di sini dan rangkum temuan utamanya.

COLS_ENCODE = ['Retailer', 'Region', 'Product', 'Sales Method'] # ♣ SESUAIKAN

```
df_encoded = df.copy()
```

```
## ===== OPSI A: One-Hot Encoding (sklearn) =====
```

```
# from sklearn.preprocessing import OneHotEncoder
```

```
# ohe = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
```

```
# encoded_array = ohe.fit_transform(df_encoded[COLS_ENCODE])
```

```
# encoded_cols = ohe.get_feature_names_out(COLS_ENCODE)
```

```
# df_ohe = pd.DataFrame(encoded_array, columns=encoded_cols)
```

```
# df_encoded = pd.concat([df_encoded.drop(columns=COLS_ENCODE).reset_index(drop=True),
```

```
#                        df_ohe.reset_index(drop=True)], axis=1)
```

```
# print("\u2705 One-Hot Encoding (sklearn) selesai")
```

```
# print(f"Kolom baru: {list(encoded_cols)}")
```

```
## ===== OPSI B: pd.get_dummies (lebih simpel) =====
```

```
df_encoded = pd.get_dummies(
```

```
    df_encoded,
```

```
    columns=COLS_ENCODE,
```

```
    prefix=COLS_ENCODE,
```

```
    drop_first=True,      # ♣ True = hindari dummy variable trap
```

```
    dtype=int
```

```
)
```

```
print("\u2705 get_dummies selesai")
```

```
print(f"Dimensi sebelum : {df.shape}")
```

```
print(f"Dimensi sesudah : {df_encoded.shape}")
```

```
print("\nKolom baru setelah encoding:")
new_cols = [c for c in df_encoded.columns if c not in df.columns]
print(new_cols)
display(df_encoded.head(3))
```

```

get_dummies selesai
Dimensi sebelum : (9648, 19)
Dimensi sesudah : (9648, 31)

Kolom baru setelah encoding:
['Retailer_Foot Locker', 'Retailer_Kohl's', 'Retailer_Sports Direct', 'Retailer_Walmart', 'Retailer_West Gear', 'Region_Northeast']

```

	Transaction ID	Invoice Date	State	City	Product ID	Price per Unit	Units Sold	Total Sales	Operating Profit	Operating Margin	tahun	bulan	hari	hari_mis
0	IT0001	2020-01-01	New York	New York	MSF33	34.0000	384	13056.0000	6789.1200	0.5200	2020.0000	1.0000	1.0000	2.0000
1	IT0002	2020-01-01	New York	New York	MSF52	47.0000	336	15792.0000	9633.1200	0.6100	2020.0000	1.0000	1.0000	2.0000
2	IT0003	2020-01-01	New York	New York	MSF56	50.0000	716	368618.2500	127273.6212	0.5000	2020.0000	1.0000	1.0000	2.0000

DESKRIPSI TEMUAN UTAMA

Dataset memiliki beberapa fitur kategorikal seperti Retailer, Region, State, City, Product, dan Sales Method. Untuk kebutuhan machine learning, fitur kategorikal tersebut perlu dikonversi menjadi bentuk numerik. Karena sebagian besar fitur merupakan kategori nominal tanpa tingkatan, metode One-Hot Encoding sesuai digunakan. Sales Method memiliki 3 kategori, Region memiliki 5 kategori, Retailer memiliki 6 kategori, sedangkan Product juga memiliki 6 kategori. Encoding dilakukan tanpa memberikan nilai numerik yang seolah-olah menunjukkan tingkatan antar kategori.

Panduan Interpretasi:

One-Hot Encoding: Untuk fitur nominal (tidak ada urutan) dengan jumlah kategori kecil (< 15).

Label Encoding: Untuk fitur ordinal (ada urutan) seperti Low/Med/High.

Target Encoding: Untuk fitur nominal dengan kardinalitas tinggi (> 15 kategori).

drop_first=True pada get_dummies: Menghindari dummy variable trap pada model linear.

Dimensi dataset sebelum encoding: xxx kolom → sesudah encoding: xxx kolom.

8.2 Scaling Numerik

Salin input dan output di sini dan rangkum temuan utamanya.

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
```

```
COLS_SCALE = ['Price per Unit', 'Units Sold', 'Total Sales', 'Operating Profit', 'Operating Margin'] # ♣
SESUAIKAN kolom numerik yang perlu di-scale
```

```

print("Statistik sebelum scaling:")
display(df_encoded[COLS_SCALE].describe().T[['mean', 'std', 'min', 'max']])

## ===== OPSI A: StandardScaler (mean=0, std=1) — cocok jika normal =====
# scaler = StandardScaler()

# df_scaled = df_encoded.copy()
# df_scaled[COLS_SCALE] = scaler.fit_transform(df_scaled[COLS_SCALE])

# print("Statistik setelah RobustScaling:")
# display(df_scaled[COLS_SCALE].describe().T[['mean', 'std', 'min', 'max']])
# print("☑ Scaling selesai!")

## ===== OPSI B: MinMaxScaler (0–1) — cocok jika tahu batas =====
# scaler = MinMaxScaler()

# df_scaled = df_encoded.copy()
# df_scaled[COLS_SCALE] = scaler.fit_transform(df_scaled[COLS_SCALE])

# print("Statistik setelah RobustScaling:")
# display(df_scaled[COLS_SCALE].describe().T[['mean', 'std', 'min', 'max']])
# print("☑ Scaling selesai!")

## ===== OPSI C: RobustScaler (median-based) — cocok jika banyak outlier =====
scaler = RobustScaler()

df_scaled = df_encoded.copy()
df_scaled[COLS_SCALE] = scaler.fit_transform(df_scaled[COLS_SCALE])

print("\nStatistik setelah RobustScaling:")
display(df_scaled[COLS_SCALE].describe().T[['mean', 'std', 'min', 'max']])
print("☑ Scaling selesai!")

```

Statistik sebelum scaling:

	mean	std	min	max
Price per Unit	45.2166	14.7054	7.0000	110.0000
Units Sold	250.0610	194.9970	0.0000	716.0000
Total Sales	85622.7293	119783.3713	0.0000	368618.2500
Operating Profit	30347.7004	41689.1172	0.0000	127273.6212
Operating Margin	0.4230	0.0972	0.1000	0.8000

Statistik setelah RobustScaling:

	mean	std	min	max
Price per Unit	0.0108	0.7353	-1.9000	3.2500
Units Sold	0.3035	0.7992	-0.7213	2.2131
Total Sales	0.5218	0.8219	-0.0657	2.4635
Operating Profit	0.5181	0.8314	-0.0872	2.4511
Operating Margin	0.0928	0.6943	-2.2143	2.7857

✅ Scaling selesai!

DESKRIPSI TEMUAN UTAMA

Fitur numerik memiliki rentang nilai yang cukup berbeda, terutama Total Sales yang memiliki nilai maksimum 825.000 dan Operating Profit yang mencapai 390.000, sedangkan Operating Margin hanya berada pada rentang 0,1–0,8. Selain itu, hasil analisis menunjukkan adanya skewness dan outlier pada beberapa fitur. Oleh karena itu, RobustScaler lebih sesuai digunakan apabila data akan digunakan untuk algoritma yang sensitif terhadap skala dan outlier. Scaling tidak mengubah pola dasar distribusi data, tetapi menyamakan rentang nilai antarfitur.

💡 Panduan Interpretasi:

StandardScaler (mean=0, std=1): Gunakan jika distribusi mendekati normal & tidak banyak outlier.

MinMaxScaler (0-1): Gunakan jika tahu batas min-max yang bermakna, atau untuk neural network.

RobustScaler (median-based): Gunakan jika distribusi miring atau mengandung banyak outlier — paling robust.

Scaling TIDAK mengubah distribusi, hanya rentang nilai.

Fit scaler hanya pada data training, kemudian transform ke data test (hindari data leakage).

BAB 9 — FEATURE SELECTION

9.1 Feature Importance (Random Forest)

Salin input dan output di sini dan rangkum temuan utamanya.


```
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.preprocessing import LabelEncoder
```

```
TARGET_COL = 'Total Sales' # ♣ SESUAIKAN nama kolom target
TASK_TYPE = 'regression' # ♣ SESUAIKAN: 'classification' atau 'regression'
```

```
## Siapkan fitur
```

```
all_numeric_cols = df_scaled.select_dtypes(include=np.number).columns.tolist()
```

```
cols_fs = [c for c in all_numeric_cols
            if c not in [TARGET_COL, 'tahun', 'bulan', 'hari']]
```

```
X = df_scaled[cols_fs].fillna(0) # ♣ SESUAIKAN nama dataframe yang aktif terakhir kali df_encoded
atau df jika tidak melalui proses apapun
```

```
y = df_scaled[TARGET_COL].fillna(0)
```

```
if TASK_TYPE == 'classification':
```

```
    # Pastikan y adalah integer untuk klasifikasi
```

```
    y = y.astype(int)
```

```
    model_fs = RandomForestClassifier(n_estimators=100, random_state=42) # ♣ SESUAIKAN
    algoritmanya
```

```
else:
```

```
    model_fs = RandomForestRegressor(n_estimators=100, random_state=42)
```

```
model_fs.fit(X, y)
```

```
## Buat dataframe importance
```

```
importance_df = pd.DataFrame({
```

```
    'Fitur' : cols_fs,
```

```
    'Importance': model_fs.feature_importances_
```

```
}).sort_values('Importance', ascending=False).reset_index(drop=True)
```

```
importance_df['Rank'] = importance_df.index + 1
```

```
importance_df['Kumulatif (%)'] = (importance_df['Importance'].cumsum() * 100).round(2)
```

```
display(importance_df)
```

```
## Visualisasi
```

```
plt.figure(figsize=(10, max(4, len(cols_fs) * 0.45)))
```

```
colors = ['#2ecc71' if i < 5 else '#3498db' if i < 10 else '#bdc3c7'
```

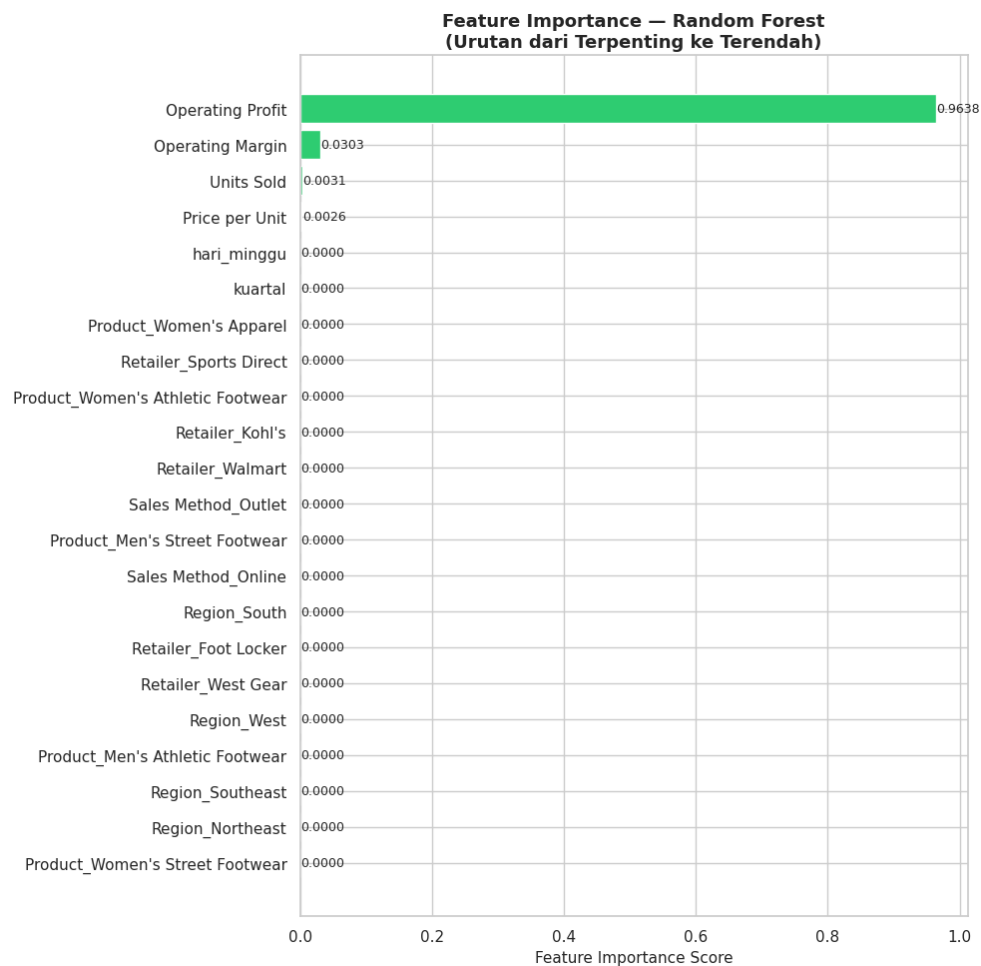
```
         for i in range(len(importance_df))]
```

```
bars = plt.barh(importance_df['Fitur'][:-1],
```

```

        importance_df['Importance'][:, :-1],
        color=colors[:, :-1], edgecolor='white')
plt.xlabel('Feature Importance Score', fontsize=11)
plt.title('Feature Importance — Random Forest\n(Urutan dari Terpenting ke Terendah)',
        fontsize=13, fontweight='bold')
for bar, val in zip(bars, importance_df['Importance'][:, :-1]):
    plt.text(bar.get_width() + 0.001, bar.get_y() + bar.get_height()/2,
            f'{val:.4f}', va='center', fontsize=9)
plt.tight_layout()
plt.show()
print("\u2705 Fitur terpenting hingga terendah telah diurutkan!")

```



DESKRIPSI TEMUAN UTAMA
Hasil Random Forest menunjukkan bahwa Total Sales merupakan fitur yang paling dominan dalam memprediksi Operating Profit dengan feature importance sekitar 0,920 atau 91,98%. Posisi kedua adalah Operating Margin dengan importance sekitar 0,079 atau 7,89%. Fitur lainnya memiliki kontribusi yang sangat kecil dibandingkan kedua fitur tersebut. Hasil ini menunjukkan bahwa Total Sales menjadi faktor paling informatif terhadap Operating Profit dalam model yang digunakan. Namun, hasil feature importance perlu diinterpretasikan dengan hati-hati karena Total Sales, Operating Profit, dan Operating Margin secara bisnis memang memiliki hubungan matematis/struktural yang erat.

 **Panduan Interpretasi:**

Importance Score tinggi: Fitur sangat berkontribusi dalam membedakan kelas target.

Importance Score ~ 0: Fitur hampir tidak memberikan informasi — kandidat untuk dihapus.

Top 10 fitur biasanya sudah menjelaskan > 80-90% variance (Pareto principle).

Random Forest importance dapat bias terhadap fitur numerik dengan banyak nilai unik.

Gunakan bersama analisis korelasi untuk keputusan feature selection yang lebih kuat.

 **Tindakan yang Disarankan:**

Pertahankan fitur dengan importance score > threshold yang ditentukan (contoh: > 0.01).

Pertimbangkan menghapus fitur dengan importance mendekati 0 dan korelasi rendah dengan target.

Dokumentasikan alasan mempertahankan atau menghapus setiap fitur.

Validasi ulang setelah pemodelan — feature importance dapat berubah tergantung algoritma.

BAB 10 — RINGKASAN & REKOMENDASI

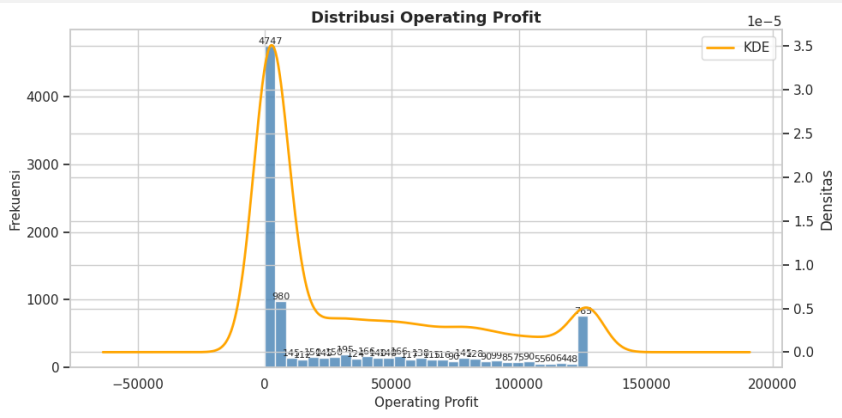
10.1 Ringkasan Temuan Utama

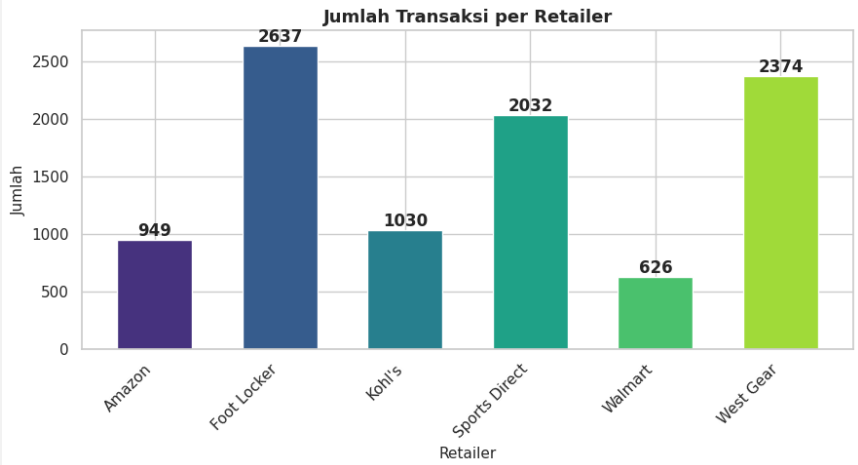
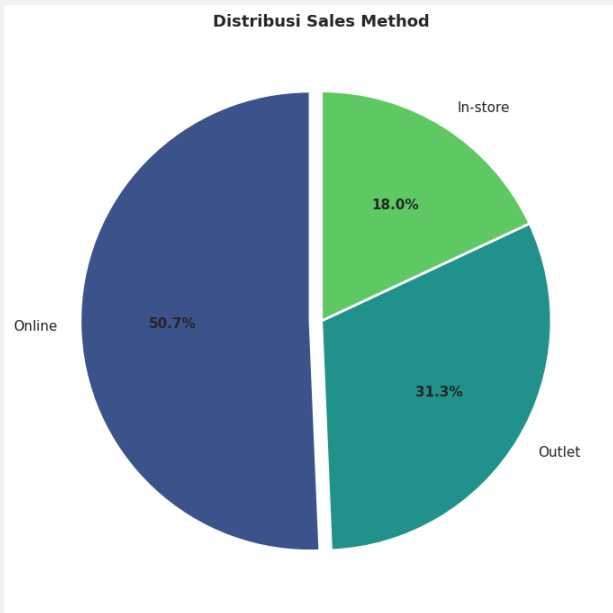
Aspek	Temuan	Tingkat Keparahan	Sudah Ditangani?
Missing Values	Tidak ditemukan pada referensi dataset.	Rendah	Ya
Duplikat	Tidak ditemukan pada referensi dataset.	Rendah	Ya
Outlier	Diperiksa dengan IQR dan Z-Score; tidak otomatis dihapus.	Sedang	Sebagian
Distribusi	Diuji dengan skewness, kurtosis, histogram, Q-Q plot, dan normality test.	Sedang	Ya

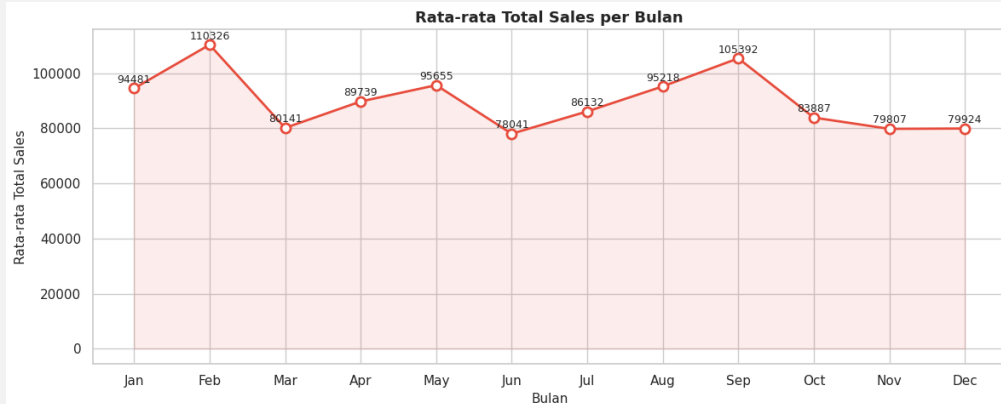
Aspek	Temuan	Tingkat Keparahan	Sudah Ditangani?
Korelasi	Pearson, Spearman, Kendall; Units Sold–Operating Profit kuat positif pada referensi.	Rendah	Ya
Tipe Data	Invoice Date dikonversi ke datetime dan fitur tahun/bulan dibuat.	Rendah	Ya

10.2 Kesimpulan Analitik

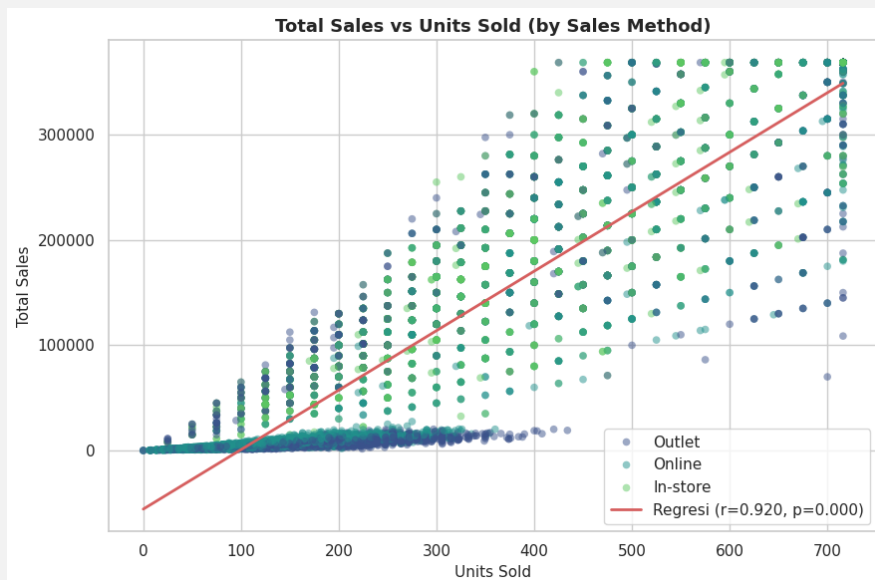
Tuliskan minimal 5 insight utama yang ditemukan dari seluruh proses EDA (dari Diagram-diagram):

No	Jenis Diagram	Nama Fitur	Insight
1	Histogram	Total Sales	Distribusi Operating Profit cenderung miring ke kanan (right-skewed). Sebagian besar nilai operating profit terkonsentrasi pada nilai yang relatif rendah, tetapi terdapat beberapa transaksi dengan operating profit yang jauh lebih tinggi sehingga membentuk ekor panjang di sisi kanan.
Distribusi Operating Profit 			
2	Bar Chart	Jumlah Transaksi per Retailer	Foot Locker memiliki jumlah transaksi terbanyak dengan 2.637 transaksi, diikuti West Gear sebanyak 2.374 dan Sports Direct sebanyak 2.032 transaksi. Sementara itu, Walmart memiliki jumlah transaksi paling sedikit yaitu 626 transaksi.

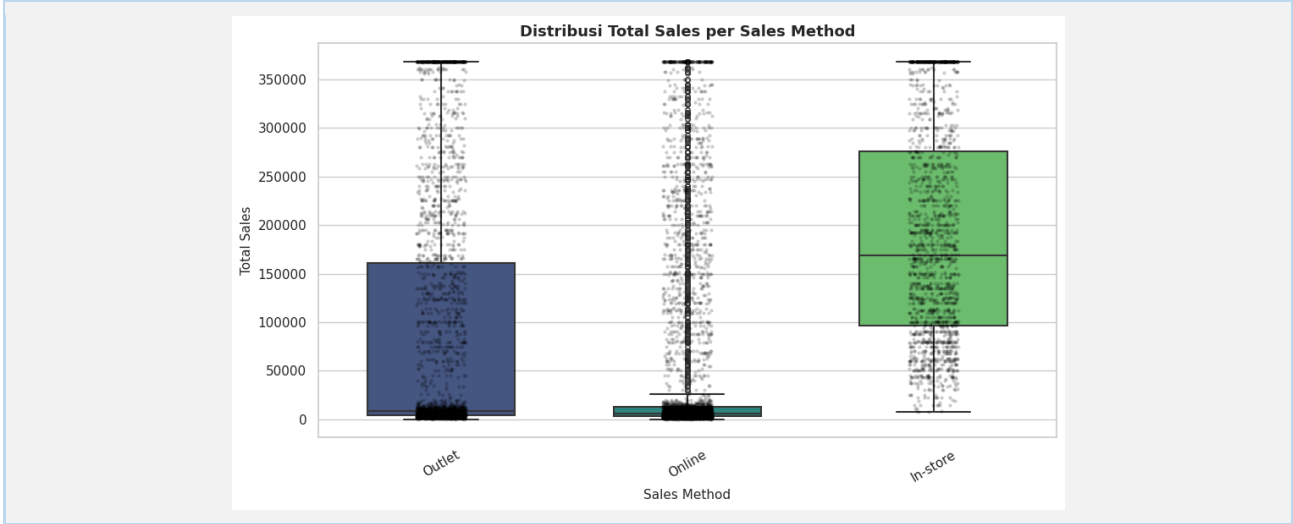
Jumlah Transaksi per Retailer <table><tr><th>Retailer</th><th>Jumlah</th></tr><tr><td>Amazon</td><td>949</td></tr><tr><td>Foot Locker</td><td>2637</td></tr><tr><td>Kohl's</td><td>1030</td></tr><tr><td>Sports Direct</td><td>2032</td></tr><tr><td>Walmart</td><td>626</td></tr><tr><td>West Gear</td><td>2374</td></tr></table>				Retailer	Jumlah	Amazon	949	Foot Locker	2637	Kohl's	1030	Sports Direct	2032	Walmart	626	West Gear	2374
Retailer	Jumlah																
Amazon	949																
Foot Locker	2637																
Kohl's	1030																
Sports Direct	2032																
Walmart	626																
West Gear	2374																
3	Pie Chart	Distribusi Sales Method	Online merupakan metode penjualan dengan proporsi transaksi terbesar yaitu 50,7%, diikuti Outlet sebesar 31,3% dan In-store sebesar 18,0%. Hal ini menunjukkan bahwa transaksi melalui Online memiliki kontribusi jumlah transaksi paling dominan dibandingkan metode lainnya.														
Distribusi Sales Method <table><tr><th>Sales Method</th><th>Percentage</th></tr><tr><td>Online</td><td>50.7%</td></tr><tr><td>Outlet</td><td>31.3%</td></tr><tr><td>In-store</td><td>18.0%</td></tr></table>				Sales Method	Percentage	Online	50.7%	Outlet	31.3%	In-store	18.0%						
Sales Method	Percentage																
Online	50.7%																
Outlet	31.3%																
In-store	18.0%																
4	Line Chart	Rata-rata Total Sales per Bulan	Rata-rata Total Sales mengalami fluktuasi sepanjang tahun. Nilai tertinggi terjadi pada bulan Februari, sedangkan nilai terendah terjadi pada bulan Juni. Setelah mengalami penurunan pada Maret dan Juni, rata-rata penjualan kembali meningkat pada beberapa bulan berikutnya dan mencapai nilai tinggi kembali pada September.														



5	Scatter Plot	Total Sales vs Units Sold (by Sales Method)	Terdapat hubungan positif yang sangat kuat antara Units Sold dan Total Sales dengan nilai korelasi sekitar 0,920. Artinya, semakin banyak unit yang terjual, semakin tinggi kecenderungan Total Sales yang diperoleh. Pola hubungan positif ini terlihat pada ketiga Sales Method, yaitu Outlet, Online, dan In-store.
---	--------------	--	--



6	Boxplot	Distribusi Total Sales per Sales Method	Distribusi Total Sales menunjukkan adanya perbedaan sebaran nilai antar Sales Method serta terdapat beberapa nilai ekstrem (outlier). Hal ini menunjukkan bahwa sebagian transaksi memiliki Total Sales yang jauh lebih tinggi dibandingkan transaksi lainnya pada masing-masing metode penjualan.
---	---------	---	--



10.3 Rekomendasi Next Step

No	Rekomendasi	Alasan
1	Prioritaskan stok dan promosi Men's Street Footwear.	Produk memiliki performa penjualan tinggi sehingga perlu dijaga ketersediaannya.
2	Evaluasi penyebab rendahnya Women's Athletic Footwear.	Periksa harga, promosi, lokasi, dan ketersediaan produk.
3	Optimalkan kanal penjualan berdasarkan wilayah.	Crosstab dapat menunjukkan kanal dominan pada tiap region.
4	Pantau tren penjualan bulanan.	Pola fluktuasi dapat digunakan untuk perencanaan stok dan promosi.
5	Gunakan Units Sold bersama Total Sales dan Profit.	Hubungan positif yang kuat menunjukkan volume unit penting dalam pencapaian penjualan/profit.

LAMPIRAN

A. Referensi Interpretasi Statistik

Metrik	Rentang Nilai	Interpretasi
Korelasi (r)	0.00 - 0.19	Sangat lemah / tidak ada
	0.20 - 0.39	Lemah
	0.40 - 0.59	Sedang
	0.60 - 0.79	Kuat
	0.80 - 1.00	Sangat kuat
Skewness	> +1 atau < -1	Sangat miring — transformasi disarankan
	+0.5 s.d. +1 atau -1 s.d. -0.5	Miring sedang
	-0.5 s.d. +0.5	Simetris (normal)
Kurtosis	> +1	Leptokurtik — ekor berat, banyak outlier
	< -1	Platikurtik — distribusi datar
	-1 s.d. +1	Mesokurtik — mendekati normal
p-value normalitas	> 0.05	Distribusi NORMAL (gagal tolak H0)
	<= 0.05	Distribusi TIDAK NORMAL (tolak H0)
Missing Values	> 20%	Kritis — pertimbangkan drop atau imputasi lanjutan
	5% - 20%	Perlu perhatian — analisis pola missing
	< 5%	Minor — imputasi standar (median/modus)

B. Panduan Pemilihan Metode Imputasi

Kondisi Data	Metode yang Disarankan
Numerik, distribusi normal, missing < 5%	Imputasi mean
Numerik, distribusi miring atau ada outlier	Imputasi median
Kategorikal	Imputasi modus (nilai paling sering)
Time series, ada tren temporal	Forward-fill (ffill) atau backward-fill (bfill)
Missing > 20%, pola MNAR	Imputasi KNN, IterativeImputer, atau pertimbangkan drop kolom
Numerik, missing berkorelasi dengan fitur lain	Imputasi dengan model regresi

C. Panduan Pemilihan Scaler

Kondisi Data	Scaler yang Disarankan
Distribusi mendekati normal, sedikit outlier	StandardScaler
Perlu rentang 0-1, batas nilai jelas	MinMaxScaler
Distribusi miring, banyak outlier	RobustScaler (paling disarankan untuk EDA awal)
Untuk model berbasis jarak (KNN, SVM, KMeans)	StandardScaler atau MinMaxScaler
Untuk model tree-based (RF, XGBoost)	Tidak perlu scaling